D3S3 CareSystem

Telugu Translation — Proposed Solution

Proposal for Team Review | March 2026

1. Executive Summary

This proposal outlines a recommended approach for implementing Telugu language support within the D3S3 CareSystem platform. The need has been identified to make the system more accessible to Telugu-speaking patients, nurses, and clinicians, and this document is intended to give the team a clear picture of the proposed solution, the reasoning behind it, and the steps that would be required to deliver it.

During the evaluation process, a HIPAA compliance issue was identified that eliminates one of the simpler options from consideration. This has naturally guided the recommendation toward a Hybrid Two-Phase approach — one that is not only compliant but is arguably the more robust and appropriate solution for a medical platform in any case.

The team is invited to review this proposal and provide input before any implementation work begins.

FOR REVIEW	*This document is presented for team discussion. No implementation decisions have been made. Feedback and questions are welcomed.*

2. Why the Google Translate Widget Is Not Suitable

The Google Translate Widget was initially considered as a zero-cost, zero-configuration option. However, after reviewing the compliance requirements for D3S3, it is not recommended and is excluded from the proposed solution. The reason is a fundamental HIPAA incompatibility that cannot be mitigated.

The Core Issue: No Business Associate Agreement

HIPAA requires that any third party handling Protected Health Information (PHI) on behalf of a covered entity must sign a Business Associate Agreement (BAA). A BAA is a legal contract that binds the third party to HIPAA-compliant data handling standards.

Google does not offer a BAA for free consumer products, including the Google Translate Widget. This is a structural limitation of the free consumer service tier, not a policy that can be

negotiated around. Transmitting any PHI through the widget without a BAA would constitute a HIPAA violation regardless of intent or context.

The Secondary Issue: No Field-Level Control

Even if a BAA were available, the widget works by sending the entire rendered page to Google's servers for translation. There is no mechanism to exclude specific fields or elements — every patient name, diagnosis, clinical note, and identifier visible on screen would be transmitted, with no ability to apply de-identification or selective processing. This makes it architecturally unsuitable for a HIPAA-regulated medical system.

HIPAA NOTE	*The Google Translate Widget cannot be made HIPAA compliant for this use case and is therefore excluded from the proposed solution.*

3. Proposed Solution: Hybrid Two-Phase Architecture

The proposed approach addresses both translation needs within D3S3 — the static interface and dynamic patient data — using two complementary layers that operate independently. Together they provide full Telugu language support while remaining HIPAA compliant and resilient to offline conditions.

Phase 1 — Static PHP Language Files (Interface Translation)

All interface elements are translated once, stored as PHP language array files on the D3S3 server, and served locally at runtime. No external service is called during normal operation.

- Covers: navigation, sidebar menu, form field labels, case sheet tab names, buttons, status indicators, validation messages, error text, system notifications
- Works fully offline — translation is served from local files with no internet dependency
- Zero ongoing cost — one-time API call to generate the files, then self-contained
- High-quality Google-grade Telugu, stored permanently as reviewed PHP arrays
- Medical terminology can be manually reviewed and corrected after generation

Example of what the language file structure looks like on the server:

English (lang/en/common.php)	Telugu (lang/te/common.php)
`'patient_name' => 'Patient Name'`	`'patient_name' => 'రోగి పేరు'`
`'appointment' => 'Appointment'`	`'appointment' => 'అపాయింట్‌మెంట్'`
`'save' => 'Save'`	`'save' => 'సేవ్ చేయండి'`

Phase 2 — Google Cloud Translation API (Dynamic Data)

Patient intake data, clinical notes, exam findings, and other free-text content entered by nurses and doctors are stored in the database as typed. When a user has Telugu selected and internet connectivity is available, selected fields are translated on-demand via the Google Cloud Translation API.

- Covers: nurse intake free-text entries, doctor clinical notes, case sheet exam findings, appointment notes, case summaries
- Does NOT cover: patient name, ID numbers, dates, structured dropdown values — these are never sent to the API
- Requires internet connectivity — degrades gracefully when offline (see Section 4)
- Google Cloud Translation API is HIPAA-eligible with a signed Business Associate Agreement (BAA)
- Google's Data Processing Addendum confirms input data is not retained or used for model training

NOTE	*Phase 2 covers translation of data entered into the system. It does not cover data entry itself — users enter data in their preferred language as they do today. Translation applies at the point of reading/display.*

4. Offline Behavior

Clinic deployments may operate in low-connectivity or offline environments. The hybrid architecture is designed so that offline conditions cause graceful degradation rather than system failure.

Feature Area	Online	Offline
UI Labels & Navigation	✓ Full Telugu from local PHP files	✓ Full Telugu — no change
Patient Intake Fields	✓ Translated via API on display	⚠ Original language shown, offline banner displayed
Doctor Clinical Notes	✓ Translated via API on display	⚠ Original language shown, offline banner displayed
Data Entry & Saving Records	✓ Full functionality	✓ Full functionality — no degradation

5. HIPAA & Legal Compliance

Patient data for D3S3 is hosted on US-based servers, making HIPAA directly applicable. This section outlines the compliance position of the proposed solution and the safeguards that would be built into the implementation.

5a. HIPAA Compliance Position

- Phase 1 static PHP files involve no PHI whatsoever — zero compliance concern
- Phase 2 Google Cloud Translation API is on Google's list of HIPAA-eligible services
- A Business Associate Agreement (BAA) must be signed with Google before any PHI is transmitted — this is a pre-launch requirement
- Google's Data Processing Addendum confirms that API input is not retained or used to train models
- The de-identification approach (see Section 5c) provides an additional protective layer even within the BAA framework

5b. Indian Law — DPDPA 2023 & SPDI Rules

As the system serves patients in India, the Digital Personal Data Protection Act 2023 and the IT Act SPDI Rules 2011 also apply:

- Health data is classified as sensitive personal data requiring explicit patient consent for third-party processing
- Transmitting clinical content to Google for translation constitutes data processing and must be disclosed in the patient consent process
- Cross-border data transfer rules (India to US-based Google servers) must comply with government-notified frameworks — these rules are still being finalized and should be reviewed by a qualified legal advisor before go-live

5c. Built-In Risk Mitigation

The following technical safeguards are built into the implementation design:

- De-identification before API calls — patient name, ID, and direct identifiers are stripped before any text is sent for translation. Clinical content is translated; patient identity is never transmitted.
- Field-level selectivity — only designated free-text clinical fields are eligible for API translation. Structured fields, demographic data, and identifier fields are excluded by design.
- Consent capture at registration — a consent acknowledgement will be added to the patient registration workflow, covering translation data processing in plain language.
- Audit logging — all translation API calls will be logged (timestamp, field type, user ID) to maintain a compliance audit trail.

- Graceful offline fallback — when connectivity is unavailable, no API call is attempted and no data leaves the system. Original-language text is displayed with a clear user indicator.

IMPORTANT	*A signed Google Cloud BAA would need to be in place before Phase 2 could go live. This is a non-negotiable pre-launch compliance requirement if the team approves this approach.*

Formal legal review by a qualified healthcare IT compliance advisor is strongly recommended before go-live, particularly regarding DPDPA cross-border transfer provisions.

6. Cost

The approved solution is designed to operate at effectively zero cost for a non-profit deployment at clinic scale.

Item	Estimated Cost	Notes
Phase 1: One-time UI string generation	**< $1 USD**	~20,000–30,000 characters of UI strings at $20 per million characters
Phase 2: Ongoing dynamic translation	$1–5 USD/month (est.)	Depends on patient volume and note length. Clinic-scale usage stays well below meaningful cost thresholds.
New Google Cloud Account Credit	**$300 USD free**	Available to new Google Cloud accounts — covers well over a year of expected usage at clinic scale

7. Implementation Plan

Should the team approve this proposal, the following actions would be required to deliver the solution. Items are sequenced in dependency order and presented for discussion.

#	Phase	Action	Priority	Owner
1	**Pre-Launch**	Create Google Cloud account to access the $300 free credit	**Critical**	Management
2	**Pre-Launch**	Sign Google Cloud Business Associate Agreement (BAA) — must be completed before any patient data is sent to the Translation API	**Critical**	Management

#	Phase	Action	Priority	Owner
3	Pre-Launch	Engage a qualified healthcare IT compliance advisor to review DPDPA cross-border data transfer provisions before go-live	Critical	Management
4	Phase 1	Design and implement i18n PHP language file structure for D3S3 codebase — folder structure, loader, language switcher wired to existing user preference	High	Dev Team
5	Phase 1	Generate Telugu translation files using Google Cloud Translation API and review all medical terminology for accuracy	High	Dev Team
6	Phase 1	Wire the existing language preference (settings page) to the new PHP language loader so the UI reflects the saved preference on login	High	Dev Team
7	Phase 2	Implement on-demand API translation layer for dynamic clinical fields with de-identification, field selectivity, and graceful offline fallback with user-facing indicator	High	Dev Team
8	Phase 2	Add patient consent acknowledgement at registration covering translation data processing	High	Dev Team
9	Phase 2	Implement audit logging for all translation API calls (timestamp, field type, user ID, success/failure)	Medium	Dev Team

8. Disclaimer

This document was prepared for internal discussion and team review. The legal and compliance information in Section 5 represents a general overview of applicable frameworks and is not a substitute for formal legal advice. Should the team approve this proposal, a formal compliance review by a qualified legal professional with expertise in Indian healthcare data law and HIPAA is strongly recommended.

9. Dev Implementation

Phase 1 — Developer Implementation Guide: i18n Infrastructure

This section provides the development team with a clear picture of what building the Phase 1 i18n infrastructure involves. It covers the folder structure, the language loader, wiring to the existing user preference, and the string extraction process.

Folder Structure

The language files should be organized under a dedicated lang/ directory at the root of the D3S3 application, with a subdirectory per language:

```
/lang
  /en
    common.php
    case_sheet.php
    appointments.php
    nurse.php
    doctor.php
    validation.php
  /te
    common.php
    case_sheet.php
    appointments.php
    nurse.php
    doctor.php
    validation.php
```

Splitting strings across multiple files by functional area rather than putting everything in one file keeps the files manageable and means only the relevant file needs to be loaded on any given page. common.php covers strings used globally across the application — navigation, buttons, status labels, and so on.

Each language file returns a simple PHP array:

```php
<?php
// lang/en/common.php
return [
    'patient_name'   => 'Patient Name',
    'date_of_birth'  => 'Date of Birth',
    'save'           => 'Save',
    'cancel'         => 'Cancel',
    'appointment'    => 'Appointment',
    'status'         => 'Status',
];
```

```php
<?php
// lang/te/common.php
return [
    'patient_name'   => 'రోగి పేరు',
    'date_of_birth'  => 'పుట్టిన తేది',
    'save'           => 'సేవ్ చేయండి',
    'cancel'         => 'రద్దు చేయండి',
    'appointment'    => 'అపాయింట్‌మెంట్',
    'status'         => 'స్థితి',
];
```

The Language Loader

A simple loader function reads the user's language preference and merges the appropriate language files into a globally accessible translation array. This should be initialized early in the application bootstrap — ideally in the existing session or config setup:

```
<?php
// includes/lang.php

function load_language(string $lang = 'en'): void {
    $supported = ['en', 'te'];
    if (!in_array($lang, $supported)) {
        $lang = 'en';
    }

    $files = ['common', 'case_sheet', 'appointments',
              'nurse', 'doctor', 'validation'];
    $strings = [];

    foreach ($files as $file) {
        $path = BASE_PATH . "/lang/{$lang}/{$file}.php";
        if (file_exists($path)) {
            $strings = array_merge($strings, require $path);
        }
    }

    $GLOBALS['_LANG'] = $strings;
}

function __(string $key, string $fallback = ''): string {
    return $GLOBALS['_LANG'][$key]
        ?? ($fallback ?: $key);
}
```

NOTE *The __() helper falls back gracefully to the key name itself if a string is missing from the Telugu file — for example during a partial rollout. This means an incomplete translation file will never break a page.*

Wiring to the Existing User Preference

The D3S3 settings page already stores a language preference per user with the value te for Telugu. The loader simply needs to be called after the session is established, passing in that saved preference:

```
<?php
// Early in bootstrap / session initialisation

require_once 'includes/lang.php';

$user_lang = $_SESSION['user_prefs']['language'] ?? 'en';
load_language($user_lang);
```

From that point on, every template in the application can call __('key') and receive the correct language output automatically. No further session checks or conditional logic are needed in individual templates.

Replacing Hardcoded Strings

The core of the Phase 1 development work is going through each PHP template and replacing hardcoded text with __() calls. For example:

```
// Before
<label>Patient Name</label>
<button type="submit">Save</button>
<span class="badge">Pending</span>

// After
<label><?= __('patient_name') ?></label>
<button type="submit"><?= __('save') ?></button>
<span class="badge"><?= __('pending') ?></span>
```

It is recommended to work through the templates in order of user-facing priority:

- Nurse staging area and patient intake forms first
- Case sheet tabs (all seven)
- Doctor queue and case assignment views
- Appointment scheduling
- Settings and administration pages

This allows Telugu to be tested end-to-end on the most critical workflows before the full replacement is complete.

String Extraction for Translation

Once all strings are replaced in the templates, the English lang/en/ files represent the complete set of strings requiring translation. These are passed to the API generation step to produce a complete Telugu file in a single run. Telugu-speaking team members then review the output rather than translating from scratch — focusing their attention on clinical terminology, medical labels, and anything that reads unnaturally in context.

Adding a new string to the application in future simply requires adding the key to the English file and its Telugu equivalent to the Telugu file. No infrastructure changes are needed.

NOTE	*It is recommended that the dev team do a rough count of distinct UI strings across D3S3 before beginning, to give the team a realistic picture of the string replacement effort involved across the full application.*